

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2
По дисциплине “Архитектура программных систем”

Выполнил, группа:
Билошицкий Михаил Владимирович, Р3316
Преподаватель:
Перл Иван Андреевич

Санкт-Петербург, 2025

Содержание

Содержание	2
Задание.....	3
Шаблон 1: Prototype (GoF)	3
Описание.....	3
Сценарии использования	3
Генерация игровых объектов:	3
Копирование конфигурации сложной формы:	4
Резервное копирование объектов:.....	4
Общие возможные ограничения	4
Шаблон 2: Singleton (GoF)	5
Описание.....	5
Сценарии использования	5
Одиночная точка доступа к настройкам приложения:.....	5
Реализация подключения к базе данных:.....	5
Логирование:	5
Общие возможные ограничения	6
Шаблон 3: Information Expert (GRASP).....	6
Описание.....	6
Сценарии использования	7
Управление заказом в интернет-магазине:	7
Расчет заработной платы сотрудников:	7
Генерация отчета о доходах:.....	7
Общие возможные ограничения	7
Шаблон 4: Creator (GRASP)	7
Описание.....	8
Сценарии использования	8
Фабрика работников:	8
Создание компонентов пользовательского интерфейса:	8
Создание заказов в интернет магазине:.....	9
Общие возможные ограничения	9
Вывод.....	9

Задание

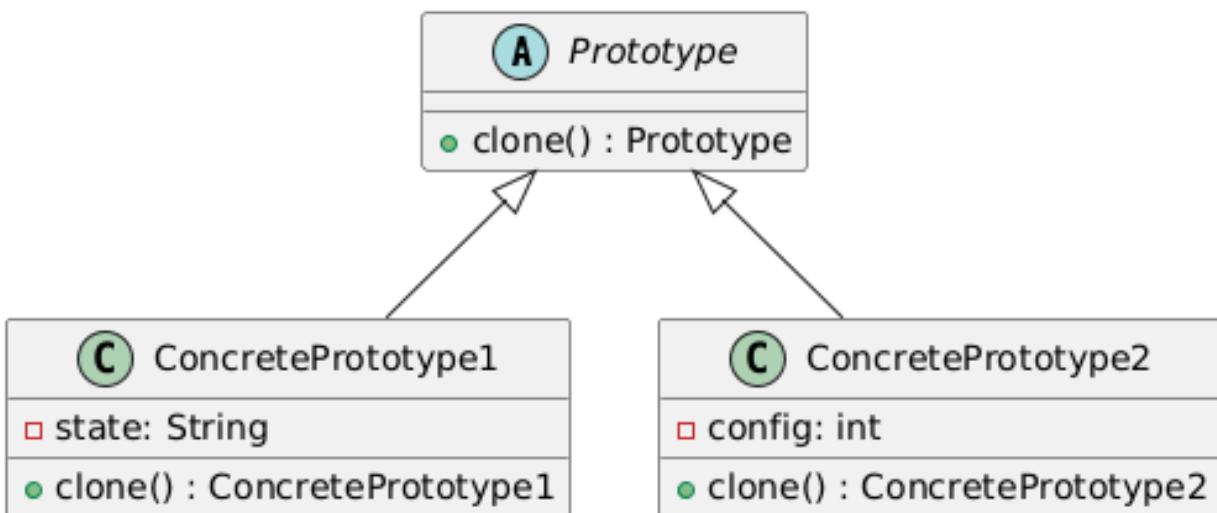
Из списка шаблонов проектирования GoF и GRASP выбрать 3-4 шаблона и для каждого из них придумать 2-3 сценария, для решения которых могут применены выбранные шаблоны.

Сделать предположение о возможных ограничениях, к которым можем привести использование шаблона в каждом описанном случае. Обязательно выбрать шаблоны из обоих списков.

Шаблон 1: Prototype (GoF)

Описание

Шаблон проектирования Prototype применяется в случаях, когда требуется создавать новые объекты путем копирования уже существующих экземпляров. Это позволяет снизить затраты на создание сложных объектов и обеспечивает возможность тонкой настройки объектов после клонирования. Он особенно полезен, если создание объектов "с нуля" является дорогостоящим процессом.



Сценарии использования

Генерация игровых объектов: В компьютерных играх может быть множество однотипных объектов, например, врагов с одинаковыми свойствами, которые периодически появляются в игре. Вместо того чтобы инициализировать их с нуля при каждом появлении, можно создать базовый объект и клонировать его. Это значительно экономит процессорное время и уменьшает вероятность ошибок.

- **Возможное ограничение:** Если объект имеет глубокую и сложную структуру с множеством ссылок на другие объекты, то при выполнении клонирования может потребоваться глубокая копия. Это увеличивает время выполнения операции и усложняет код.

Копирование конфигурации сложной формы: В графических редакторах пользователи часто копируют элементы (например, фигуры, текстовые поля, линии). Если каждый элемент имеет сложную внутреннюю структуру, то использование Prototype для клонирования их состояния позволяет упростить процесс копирования и настройки.

- **Возможное ограничение:** В случае изменения структуры исходного объекта (например, добавления новых свойств) копирование может нарушиться, если соответствующим образом не обновить механизм клонирования (особенно при использовании ручных методов клонирования).

Резервное копирование объектов: В системе резервного копирования требуется временно клонировать состояние текущего объекта для хранения в памяти или записи в файловую систему. Prototype позволяет легко дублировать объекты с сущностями, состояние которых может быть изменчивым.

- **Возможное ограничение:** Проблемы могут возникнуть при клонировании объектов, содержащих ссылки на внешние ресурсы, такие как файлы или сетевые соединения. Такие зависимости придется перерабатывать, чтобы исключить конфликт из-за дублирования активных ресурсов.

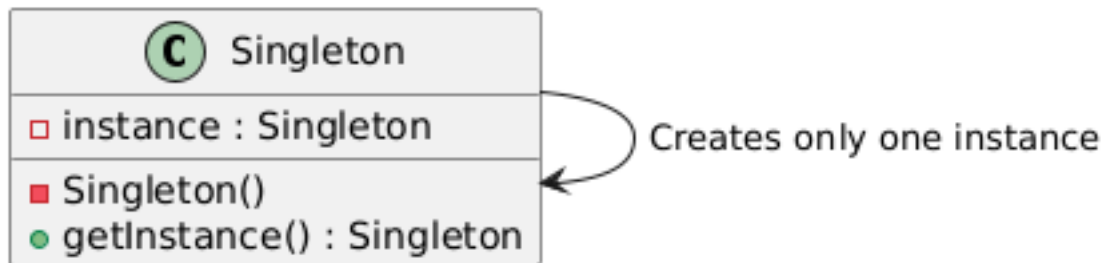
Общие возможные ограничения

- Над каждым прототипом требуется тщательно работать, чтобы убедиться, что копия будет независимой от оригинала. Ошибки в процессе клонирования могут быть трудноуловимыми.
- Для глубокого копирования сложных объектов (например, с большим числом вложенных структур или ссылок) может потребоваться значительное количество ресурсов.
- Поддержка клонирования увеличивает сложность кода, особенно если объект содержит зависимости, требующие разрешения.

Шаблон 2: Singleton (GoF)

Описание

Singleton гарантирует, что класс имеет только один экземпляр, и предоставляет глобальную точку доступа к этому экземпляру. Обычно используется для управления доступом к состоянию, которое должно быть единым для всех объектов приложения.



Сценарии использования

Одиночная точка доступа к настройкам приложения: В больших системах настройки приложений часто хранятся в одном месте. Singleton обеспечивает удобный доступ к объекту настроек, гарантируя, что одновременно существует только одна его копия.

- **Возможное ограничение:** Использование глобальных объектов делает код менее модульным, так как части приложения становятся зависимыми от единого глобального объекта.

Реализация подключения к базе данных: Приложение может использовать один объект подключения для взаимодействия с базой данных. Singleton используется для создания, управления подключением и предотвращения случайного создания множества соединений.

- **Возможное ограничение:** Если приложениям требуется работа с несколькими подключениями (например, при использовании реплицированных или шардированных баз данных), Singleton ограничивает гибкость архитектуры.

Логирование: При проектировании систем логирования Singleton используется для гарантии, что все компоненты приложения пишут логи через один экземпляр логера. Это упрощает управление логами и предотвращает разнобой.

- **Возможное ограничение:** Потенциальные проблемы с многопоточностью, если несколько потоков одновременно записывают логи через Singleton, и механизм записи не защищен синхронизацией. Это может привести к повреждению данных в логах.

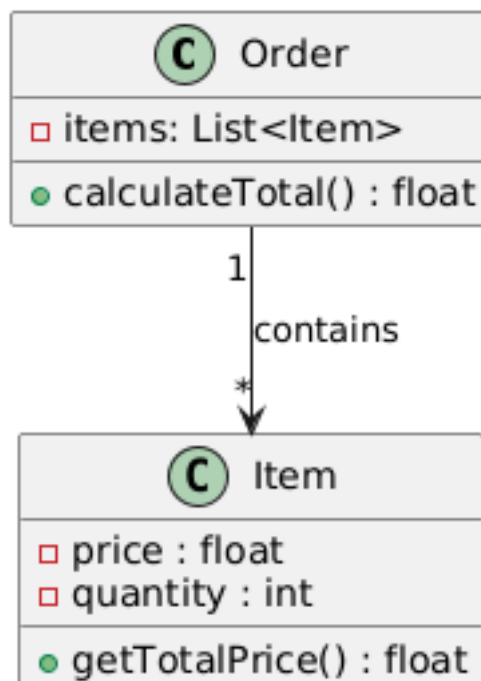
Общие возможные ограничения

- Нарушение принципа единственной ответственности, так как Singleton может стать универсальной точкой хранения данных, что трудно тестировать и отлаживать.
- Создание сильных зависимостей между различными компонентами приложения, что усложняет масштабирование.
- Проблемы с многопоточностью могут возникнуть при реализации Singleton, особенно в случае ленивой инициализации.

Шаблон 3: Information Expert (GRASP)

Описание

Information Expert предполагает, что ответственность за выполнение операций должна быть возложена на тот объект, который обладает наибольшей информацией, необходимой для выполнения этой операции. Шаблон минимизирует передачу необходимых данных между объектами, а также упрощает реализацию и тестирование



Сценарии использования

Управление заказом в интернет-магазине: В системе интернет-магазина класс Order должен содержать логику для подсчета общей стоимости заказа. Поскольку этот класс уже обладает информацией о всех товарах и их количестве, логично перенести ответственность за вычисление стоимости именно сюда, а не вызывать внешние сервисы.

- **Возможное ограничение:** Если логика подсчета общей стоимости включает сложные правила, это может сделать объект "Order" слишком перегруженным ответственностью, усложняя его поддержку.

Расчет заработной платы сотрудников: Если система управления кадрами содержит объект "Сотрудник" (Employee), расчет заработной платы должен быть его ответственностью. Это связано с тем, что объект Employee знает все, что требуется для расчета: часы, отработанные сотрудником, его тарифную ставку, льготы и бонусы.

- **Возможное ограничение:** Если объект "Employee" становится ответственным за выполнение сложных операций, требующих взаимодействия с другими компонентами (например, доступ к данным из внешней базы), это нарушает принцип единой ответственности (SRP).

Генерация отчета о доходах: Объект "Компания" (Company) может владеть всеми объектами, представляющими сотрудников. Именно объект компании предоставляет агрегированные данные о доходах всех сотрудников и организует их отображение в отчете.

- **Возможное ограничение:** Перекалывание ответственности на объект "Компания" может привести к нарушению принципа разделения логики, если этот класс станет выполнять как бизнес-логику, так и подготовку данных для отчета.

Общие возможные ограничения

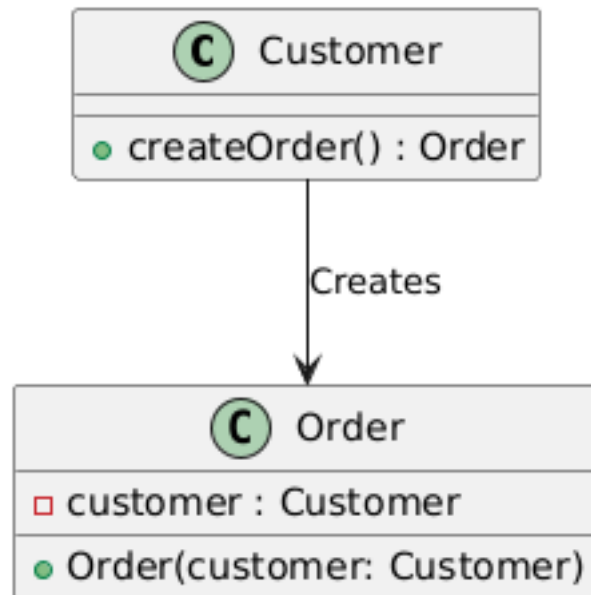
- Шаблон может привести к созданию "больших" классов, перегруженных ответственностями.
- Зависимость от внутренней структуры объекта может затруднять рефакторинг, тестирование и расширение системы.

Шаблон 4: Creator (GRASP)

Описание

Creator определяет класс, который должен быть ответственен за создание экземпляров других классов. Класс, отвечающий за создание объекта, обычно является:

- Классом, содержащим или агрегирующим создаваемые объекты.
- Классом, использующим создаваемые объекты.
- Классом, который обладает достаточной информацией для их создания.



Сценарии использования

Фабрика работников: Предположим, существует система управления кадрами предприятия. Класс `HRDepartment` может отвечать за создание объектов "Сотрудник" (`Employee`), так как этот класс знает все данные, нужные для создания сотрудника и инициализации его свойств.

- **Возможное ограничение:** Если добавляются новые типы сотрудников с уникальными характеристиками, это может потребовать изменения логики класса `HRDepartment`. Это нарушает принцип открытости/закрытости (ОСР).

Создание компонентов пользовательского интерфейса: В графическом приложении класс `WindowManager` может отвечать за создание окон, диалогов и кнопок, так как он управляет основным жизненным циклом компонентов.

- **Возможное ограничение:** Если создание UI-компонентов становится гибким и требует множественных параметров, это может привести к сильной зависимости между WindowManager и всеми типами UI-объектов, усложняя поддержку.

Создание заказов в интернет магазине: Объект Customer может выступать создателем заказов (Order), так как он обладает нужной информацией о клиенте, необходимой для начальной инициализации заказа.

- **Возможное ограничение:** Если в будущем добавится необходимость создания заказов из разных источников (например, через API или другими системами), то привязка создания только к объекту Customer становится ограничивающим фактором.

Общие возможные ограничения

- Может возникнуть высокая зависимость "создателя" от создаваемого объекта, что нарушает принцип инверсии зависимостей (DIP).
- Если объектам требуется сложная и гибкая инициализация, использование Creator может затруднить расширение системы.

Вывод

Каждый из описанных шаблонов проектирования обладает как преимуществами, так и ограничениями. Их внедрение требует детального анализа архитектуры системы, чтобы подобрать наиболее подходящий шаблон для решения конкретной задачи. Ключевая цель при выборе шаблона — найти баланс между простотой реализации, гибкостью и удобством сопровождения системы. Применение шаблонов должно быть обусловлено специфическими требованиями проекта и учитывать возможное влияние на дальнейшее развитие и поддержку системы.